

Mutation testing: Mutation operators for source code

Meenakshi D'Souza

International Institute of Information Technology
Bangalore.

October 18, 2021

Mutation operators for software artifacts

- Mutation operators are available for several programming languages including C, Ada, Java, Fortran, Lisp etc.
- Operators for C integration testing and Java class relationships are also available.
- Formal specification languages like SMV, mark-up languages like XML also have mutation operators.
- Several of these languages have a large set of mutation operators that overlap, but, some differences are particular to the language.

Designing mutation operators

- Mutation operators are designed to mimic typical programmer mistakes,
  - Operators that change relational operators or variable references
- They are also designed to encourage testers to follow common testing heuristics.
  - `FailOnZero()` that we saw in the last lecture.
- Mutation operators are designed as an exhaustive set. Effective mutation operators can be picked from this set.
  - Tests need to be created to kill the mutants.

Effective mutation operators

- Effective mutation operators: If tests that are created specifically to kill mutants created by a collection of mutation operators $O = \{o_1, o_2, \ldots\}$ also kill mutants created by all remaining mutation operators with very high probability, then O defines an effective set of mutation operators.
- It is difficult to find an effective set of mutation operators for a given program.
- Empirical studies have indicated that mutation operators that insert unary operators and those that modify unary and binary operators will be effective.

Program-level mutation operators

We provide a list of program level mutation operators that deal with unary and binary, relational and arithmetic operators. They can be applied to many programming languages.

- Absolute Value Insertion: Each arithmetic expression (and sub-expression) is modified by the functions `Abs()`, `negAbs()` and `failOnZero()`.
  - `Abs()` returns the absolute value of the expression.
  - `negAbs()` returns the negative of the absolute value.
  - `failOnZero()` tests whether the value of the expression is zero. If it is, the mutant is killed. Otherwise, execution continues and the value of the expression is returned.
- E.g.: The statement "`x=3*a`" is mutated to create the following three statements:
  `x=3*abs(a)`, `x=3*negAbs(a)` and `x=failOnXero(a)`.

Program-level mutation operators

- Arithmetic Operator Replacement: Each occurrence of one of the arithmetic operators +, -, *, /, ** and % is replaced by each of the other operators.
- In addition, each is replaced by the special mutation operators *leftOp*, *rightOp* and *mod*.
  - *leftOp* returns the left operand (the right is ignored).
  - *rightOp* returns the right operand.
  - *mod* computes the remainder when the left operand is divided by the right.
- E.g.: The statement `x = a+b` is mutated to create the following seven statements:
  `x = a-b`, `x = a*b`, `x = a/b`, `x = a**b`, `x = a`, `x = b` and `x = a % b`.

Program-level mutation operators

- Relational Operator Replacement: Each occurrence of one of the relational operators ($<$, $>$, $\leq$, $\geq$, $=$, $\neq$) is replaced by each of the other operators and by *falseOp* and *trueOp*.
  - *falseOp* always returns false and *trueOp* always returns true.
- E.g.: The statement `if (m>n)` is mutated to create the following seven statements:
  `if (m≥n)`, `if (m<n)`, `if (m≤n)`, `if (m==n)`, `if (m=n)`, `if (false)`, `if (true)`.

NPTEL

Program-level mutation operators

- Conditional Operator Replacement: Each occurrence of each logical operator (and-&&, or-||, and with no conditional evaluation- &, or with no conditional evaluation- |, not equivalent-^) is replaced by each of the other operators. In addition, each is replaced by *falseOp*, *trueOp*, *leftOp* and *rightOp*.
  - *leftOp* returns the left operand (the right is ignored) and *rightOp* returns the right operand (the left is ignored).
- E.g.: The statement `if (a && b)` is mutated to create the following eight statements:
  `if (a|||b)`, `if (a&b)`, `if (a|b)`, `if (a^b)`, `if (false)`, `if (true)`, `if (a)`, `if (b)`.

Program-level mutation operators

- Shift Operator Replacement: Each occurrence of one of the shift operators <<, >> and >>> is replaced by each occurrence of the other operators. In addition, each is replaced by the special mutation operator *leftOp*.
  - *leftOp* returns the left operand unshifted.
- E.g.: The statement `x = m << a` is mutated to create the following three statements:

  `x = m >> a`, `x = m >>> a`, `x = m`.

NPTEL

Program-level mutation operators

- Logical Operator Replacement: Each occurrence of each bitwise logical operator (*bitwise and* (&), *bitwise or* (||) and *exclusive or* (^)) is replaced by each of the other operators. In addition, each is replaced by *leftOp* and *rightOp*.
  - *leftOp* returns the left operand (right is ignored) and *rightOp* returns the right operand (left is ignored).
- E.g.: The statement `x = m & n` is mutated to create the following four statements:
  `x = m || n`, `x = m ^ n`, `x = m` and `x = n`.

NPTEL

Program-level mutation operators

- Assignment Operator Replacement: Each occurrence of one of the assignment operators (+=, -=, *=, =, %=, &=, ||=, ^=, <<=, >>=, >>>=) is replaced by each of the other operators.
- E.g.: The statement `x += 3` is mutated to create the following ten statements:
  `x-=3`, `x*=3`, `x=3`, `x%=3`, `x&=3`, `x||=3`, `x ^ =3`, `x<<=3`, `x>>=3`, `x>>>=3`.

NPTEL

Program-level mutation operators

- Unary Operator Insertion: Each unary operator (arithmetic $+$, arithmetic -, conditional , logical $\sim$) is inserted before each expression of the correct type.
- E.g.: the statement `x = 3 * a` is mutated to create the following four statements:

  `x = 3 * +a`, `x = 3 * -a`, `x = +3 * a`, `x = -3 * a`.

NPTEL

Program-level mutation operators

- Unary Operator Deletion: Each unary operator (arithmetic +, arithmetic -, conditional , logical $\sim$) is deleted.
- E.g.: the statement `if (a > -b)` is mutated to create the following two statements:
  `if (a > -b)`, `if (a > b)`.

NPTEL

Program-level mutation operators

- Scalar variable replacement: Each variable reference is replaced by every other variable of the appropriate type that is declared in the current scope.
- E.g.: the statement `x = a * b` is mutated to create the following six statements:
  `x = a * a`, `a = a * b`, `x = x * b`, `x = a * x`, `x = b * b`, `x = a * b`.

NPTEL

Program-level mutation operators

- Bomb Statement Replacement: Each statement is replaced by a special `Bomb()` function.
- `Bomb()` signals a failure as soon as it is executed, thus requiring the tester to reach each statement.

NPTEL

Program level mutation operators: Summary

- We saw a reasonably exhaustive listing of program level mutation operators that can be applied for many programming languages, with small differences.
- They can be applied during unit testing to mimic typical programmer mistakes.
- Next lecture: Comparing mutation operators with other coverage criteria.

NPTEL